

Introduction to Blockchain and Smart Contracts

Lecture 5: Nakamoto Consensus

Subodh Sharma | IIT Delhi | Jan 22, 2026



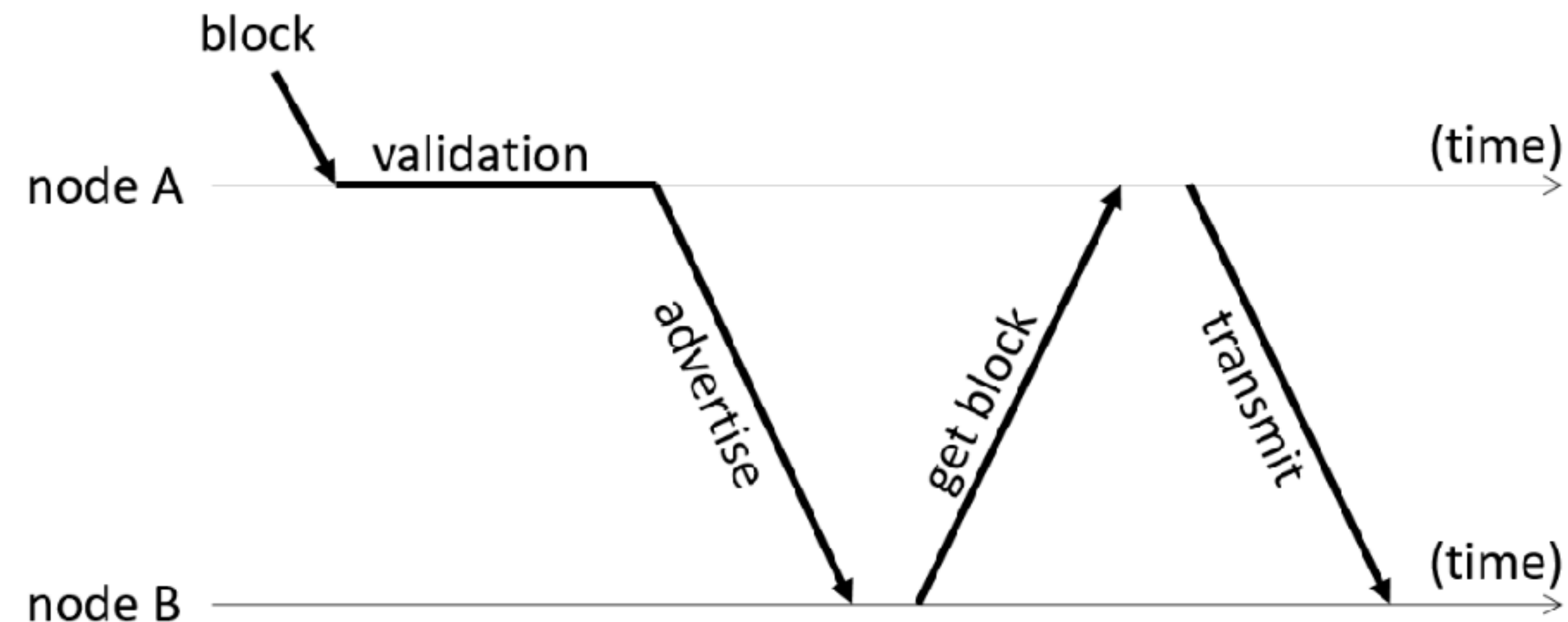


Nakamoto Consensus

- **Network Model:** Peer-to-peer overlay n/w over internet; every node runs the Bitcoin protocol and maintains a replica of Blockchain
 - That is: **asynchronous message-passing system** with bounded transmission delay
 - Additionally, **permissionless access** and best-effort information propagation
 - **Permissionless Access:**
 - Fetch a list of initial peers from known DNS servers; search for new peers by either asking DNS servers or by listening to msgs from other peers; retrieve a blockchain replica from peers and start with the operations

Nakamoto Consensus

- **Information Propagation:**
 - **Reading Assignment:** Christian Decker and Roger Wattenhofer. Information propagation in the bitcoin network.





Nakamoto Consensus Protocol

- Each node runs the **same protocol**
- Each node manages its replica of blockchain **independently**
- **Security:** Depends on majority of nodes being honest
- **Protocol rules:**
 - All newly **received** or **generated** blocks must be bcast to peers in a **timely manner**
 - Blocks and transactions must be validated before bcast
 - **Longest chain is always the desired chain;**



Nakamoto Consensus Protocol

- **What does it solve:** single ordered history of transactions
- **Protocol workflow:**
 - User creates a txn and signs it then bcasts (**gossip** protocol) it
 - Nodes **validate** it (check signature, no double spend, etc.) and put in their **mempool**
- **A miner's workflow:**
 - Selects subset of mempool txns and creates a **candidate block**
 - Candidate block: prev block hash + merkle root of the txns + **coinbase** txn
 - Searches for a nonce such that PoW challenge is successfully met



Nakamoto Consensus Protocol

- **Block Propagation & Acceptance:** Node receives a new block
 - Check if PoW is valid & difficult target matches rules
 - Check all transactions in the block are valid
 - Check if the block extends the node's **best chain: yes, then add it**
 - If the block extends some earlier block, it will create a fork
 - Store it and compare the cumulative work of competing branches
 - Switch to the branch with most work, if needed



Nakamoto Consensus Protocol

- **Fork resolution:**
 - Miners A and B find blocks at the same time
 - Some nodes see A's block first and others see B's block first
 - Miner's mine on the blocks they see first (in subsequent rounds)
 - The next found block will extend one branch, making it heavier
 - Nodes converge to the heavier branch
 - **confirmation depth** matters



Nakamoto Consensus Protocol

- **Consensus in Nakamoto Consensus:**
 - Probabilistic (strengthens when more blocks are built on top of a txn)
 - Honest miners being in majority will extend the honest chain faster than attackers
- Liveness (progress) & Safety (consistency):
 - **Liveness:** there is some miner power online and the n/w is not always partitioned
 - **Safety:** deeper the confirmation depth — probabilistic



Nakamoto Consensus Protocol

- Longest chain rule:

```
Node A: Block 0 --> Block 1 --> Block 2 --> Block 3 --> Block 4
                                         \
Node B:                               --> Block 3' --> Block 4'
```

- Scenario 1: if Block 4 is mined before Block 4' in the example above, Node B will discard its shorter chain (Block 3' -> Block 4') and switch to the longer chain (Block 3 -> Block 4).
- Scenario 2: If Block 4' is mined before Block 4, and then Block 5 is added to the chain starting from Block 3', all nodes will eventually accept this as the valid chain since it is now the longest.



Nakamoto Consensus Protocol

- **Proof of Work:**
 - Block generation requires inserting a nonce in the header.
 - The hash of the header is less than a particular value
- **Probabilistic Finality:**
 - A block not on the longest chain can be discarded
 - Let $p (\leq 50\%)$ denote the hashing power of an adversary, then the probability of an attacker to catch up from m blocks behind is:
 - $P(\text{catch-up}) = (p/1 - p)^m$